CSE 2231 MIDTERM 2 QUESTION BANK

Autumn 2026 • Questions from the course schedule, homework, projects, practice midterm, and supplied project materials.

- 125 multiple-choice questions
- 56 short-response questions
- 18 topic groups
- Answer key with explanations and model responses included
- Study scope: meetings 1–41, through Friday, November 6, 2026

How to use this book

Work through Part I and Part II without looking ahead. Part III contains all multiple-choice answers and explanations. Part IV contains short-response model answers. Pair this book with the companion local Java practice app for coding challenges.

All questions are original practice questions based on the covered concepts and your files; they are not predictions or copies of the actual exam.

Coverage map

Each row is linked to your course topics. Material from before Midterm 1 is included because it is a prerequisite to Midterm 2 projects.

Topic	MC	Short
Contracts & Mathematical Models	8	3
Sequences, Smooth & Arithmetic	6	3
Kernel Implementations & Testing	8	3
Stacks, Queues & Layering	6	3
P01 WordCounter	5	2
P02 NaturalNumber on String	5	2
Hashing & Map4	8	4
Trees & Recursive Traversal	7	3
Binary Search Trees & P04	6	3
Sorting & Comparators	7	3
Heaps, Heapsort & P05	8	4
Linked Lists, Iterators & P06	9	3
BugsWorld & BL Programs	4	2
Statement ASTs & Recursion	9	4
Program Kernels & Refactoring	6	3
Grammars, Tokenizer & Recursive Descent	11	6
Projects & Debugging Scenarios	6	3
Exam-Style Tracing & Complexity	6	2

Primary references: https://cse22x1.engineering.osu.edu/2231/schedule.html ; the uploaded Homework 13–28 PDFs, three project archives, and Autumn 2026 practice midterm/key.

Part I — Multiple Choice

Circle one best answer for each question. Explanations are located in Part III.

Contracts & Mathematical Models

M001. In a contract, what does #s represent?

A. The size of s
B. The incoming value of s
C. The successor of s
D. The previous loop iteration

M002. What is the parameter mode @replaces q?

A. q is unchanged
B. q is consumed and empty
C. q may be overwritten with a specified outgoing value
D. q is read-only

M003. The mathematical model of a Queue is a…

A. set
B. string/sequence
C. partial function
D. binary tree

M004. Which mathematical model fits a Map<K,V>?

A. a total function
B. an unordered set of K-V pairs forming a partial function
C. an ordered sequence of keys
D. a multiset of values

M005. For a method with @requires |s| >= 1, calling it with an empty s is…

A. valid and returns null
B. a violation of its precondition
C. guaranteed to throw IllegalArgumentException
D. equivalent to clear

M006. The @clears parameter mode means…

A. the parameter must be null

B. the outgoing parameter has its type’s initial value

C. the object is destroyed

D. the incoming value is preserved

M007. What does perms(result, #input) express?

A. They have equal capacity

B. They contain the same entries with multiplicities, possibly reordered

C. The resulting data is sorted

D. The objects have the same identity

M008. The contract s1 * s2 means…

A. multiplication

B. set union

C. sequence concatenation

D. comparison

Sequences, Smooth & Arithmetic

M009. smooth(<2,4,6>) must produce…

A. <3,5>

B. <2,3,5>

C. <2,4,6>

D. <4,6>

M010. For a one-element input <7>, smooth returns…

A. <7>

B. <0>

C. <>

D. undefined due to precondition

M011. Which test is essential for a correct integer average routine?

A. Only positive equal numbers

B. Only nonzero values

C. MIN_VALUE and MAX_VALUE

D. Only multiples of two

M012. Why is return (j + k) / 2 unsafe for int average?

A. Java int does not have /

B. The sum can overflow before division

C. Addition is not associative

D. Division always rounds up

M013. The value of -3 / 2 in Java is…

A. -2

B. -1

C. 1

D. 2

M014. A useful boundary-focused JUnit test suite includes…

A. only ordinary inputs

B. only one random case

C. empty, singleton, repeated, and extreme inputs where permitted

D. only sorted inputs

Kernel Implementations & Testing

M015. A kernel method differs from a secondary method because…

A. kernel methods are always faster

B. secondary methods can be implemented by layering on kernel methods

C. kernel methods cannot modify objects

D. secondary methods are private

M016. What establishes an implementation’s representation invariant?

A. the @convention clause

B. the @correspondence clause

C. the JUnit class name

D. the size method only

M017. The @correspondence clause describes…

A. what Java imports are needed

B. how the private representation denotes the abstract value

C. the object hash code

D. which tests to run

M018. In a Queue represented with a Sequence in the same order, dequeue should remove index…

A. 0
B. length-1
C. length/2
D. arbitrary

M019. The initial value returned by newInstance is…

A. a deep copy of this
B. a new object of the same dynamic type at initial value
C. always null
D. always a Stack

M020. After dest.transferFrom(src), the source is…

A. a copy of dest
B. at its initial value
C. undefined/invalid
D. unchanged

M021. What does test-plan partitioning primarily accomplish?

A. prove no bug exists
B. cover behavior categories and boundaries systematically
C. avoid assertions
D. ensure O(1) time

M022. When an assert inside add invokes hasKey and fails, what may actually be defective?

A. only add
B. hasKey or add or the test precondition
C. only the compiler
D. only equals

Stacks, Queues & Layering

M023. A stack follows…

A. FIFO
B. LIFO
C. random removal
D. lexical order

M024. A queue follows…

A. LIFO

B. FIFO

C. binary search order

D. sorted order

M025. A Sequence implemented with two stacks moves entries to change the cursor by…

A. swapping stack references only

B. popping one and pushing the other

C. sorting both stacks

D. calling hashCode

M026. A queue front secondary method may use dequeue then enqueue and rotation to…

A. destroy the front

B. observe front while restoring logical order

C. reverse the queue

D. sort the queue

M027. In a singly linked Stack with a top pointer, push and pop normally take…

A. O(1) each

B. O(n) each

C. O(log n)

D. O(n log n)

M028. Which is NOT permitted by Set semantics?

A. Removing an existing element

B. Adding an element not present

C. Storing duplicates as distinct set entries

D. Testing contains

P01 WordCounter

M029. To count words case-insensitively, normalize tokens by…

A. hashing the full line

B. converting tokens to lowercase before Map lookups

C. sorting the original file

D. removing vowels

M030. The WordCounter map associates…

A. file offsets with characters

B. word strings with integer counts

C. word count with filename

D. HTML tags with booleans

M031. Why copy map keys into a Queue before producing alphabetical HTML rows?

A. Map iteration order is guaranteed alphabetical

B. Queue.sort can order the words explicitly

C. Maps are immutable

D. Sorting HTML is impossible

M032. Which token should a word-counter skip?

A. “hello”

B. “World”

C. “ ”

D. “software”

M033. A bug if HTML output directly inserts raw user tokens without escaping is…

A. guaranteed compilation failure

B. the HTML can be malformed or interpret markup

C. counts become negative

D. sorting becomes quadratic

P02 NaturalNumber on String

M034. In your uploaded NaturalNumber3 representation, zero is stored as…

A. "0"

B. ""

C. null

D. "00"

M035. multiplyBy10(k) on a nonzero string representation should…

A. prepend k

B. append digit k

C. sort digits

D. parse into int first

M036. divideBy10() must return…

A. the first digit

B. the final digit as integer

C. the new length

D. the tens digit

M037. Why should zero.multiplyBy10(0) leave an empty representation?

A. to preserve the zero convention

B. because append is forbidden in Java

C. to avoid NullPointerException

D. because RADIX is 0

M038. Which is the required effect of transferFrom on the source?

A. source unchanged

B. source reset to initial zero

C. source doubles

D. source loses its type

Hashing & Map4

M039. To convert a possibly negative hash to a bucket in [0,n), use…

A. hash % n only

B. Math.floorMod(hash,n) or adjusted remainder

C. hash / n

D. Math.abs(hash) always

M040. A hash collision occurs when…

A. two equal keys have different values

B. distinct keys map to the same bucket

C. a Map is empty

D. hashCode returns a String

M041. Map4’s representation is primarily…

A. an array of Map buckets

B. a linked list of Stack objects

C. one giant sorted array

D. a binary heap

M042. Why initialize every Map4 bucket in createNewRep?

A. null buckets are valid maps

B. to preserve the non-null bucket invariant

C. to make length negative

D. to forbid collisions

M043. In Map4 removeAny, the algorithm first…

A. finds any nonempty bucket

B. sorts all keys

C. hashes a missing key

D. clears the table

M044. What is the major reason to choose a prime number of buckets?

A. primes guarantee no collisions

B. primes may reduce patterned collisions with some hash distributions

C. primes make modulo free

D. primes guarantee O(1) worst case

M045. Given Java remainder -7 % 5 = -2, mathematical mod(-7,5) is…

A. -2

B. 2

C. 3

D. 7

M046. For a Map bucket index, what property must equal keys have?

A. different hash codes

B. equal hash codes

C. different bucket indexes

D. same value object

Trees & Recursive Traversal

M047. A BinaryTree node has at most…

A. one child

B. two children

C. three children

D. arbitrarily many children

M048. The size of a nonempty tree recursively equals…

A. 1 + sum of child sizes

B. 1 + maximum child size

C. height squared

D. two times the root

M049. The height of a nonempty general Tree using the HW21 convention is…

A. number of leaves

B. 1 + maximum child height

C. sum of all child heights

D. size / 2

M050. For a BinaryTree recursive method using disassemble, what must happen before returning?

A. leave the tree empty

B. assemble the same root and children back

C. append child labels to root

D. call clear

M051. Prefix tree representation for an empty BinaryTree is…

A. null

B. <>

C. ()

D. {}

M052. A leaf with label a has the specified prefix representation…

A. a

B. a(()())

C. (a)

D. a()

M053. Which recursion avoids exploring an entire BST for a single search key?

A. search both children unconditionally

B. search only subtree selected by root comparison

C. always go right

D. only inspect leaves

Binary Search Trees & P04

M054. In a BST without duplicates, left subtree values are…

A. less than the root

B. greater than the root

C. all equal to root

D. unordered

M055. To find the inorder successor of a node with a right subtree, take…

A. largest node of right subtree

B. smallest node of right subtree

C. left child only

D. root of entire tree

M056. A degenerate unbalanced BST has worst-case search time…

A. O(1)

B. O(log n)

C. O(n)

D. O(n log n)

M057. Inserting 50,30,70,20 into an empty unique-key BST makes 20…

A. root

B. left child of 30

C. right child of 70

D. right child of 30

M058. P04 Set-on-BST implements Set membership by…

A. linear scanning all nodes

B. following comparisons through BST branches

C. hashing into a queue

D. sorting each query

M059. Which method removes the replacement when deleting a BST node with two children?

A. removeSmallest(right subtree)

B. enqueue(root)

C. heapify(tree)

D. removeAny() on unrelated set

Sorting & Comparators

M060. The contract IS_TOTAL_PREORDER for sorting relies on…

A. a comparator defining reflexive, transitive, total ordering relation

B. random order

C. set uniqueness

D. hashCode order

M061. Insertion sort repeatedly…

A. partitions around pivots

B. inserts each extracted element into a sorted prefix/queue

C. heapifies only once

D. compares all pairs in parallel

M062. In insertion sort on a Queue, insertInOrder must preserve…

A. only length

B. sorted order and permutation

C. the original front unchanged

D. every index value

M063. Quicksort partitions entries relative to…

A. the queue size

B. a chosen pivot

C. the last hash bucket

D. tree height

M064. Quicksort’s average and worst time complexities are…

A. O(n), O(n)

B. O(n log n), $O(n^2)$

C. O(log n), O(n)

D. $O(n^2)$, O(n log n)

M065. If compare(a,b) returns a negative number, that convention means…

A. a comes before b

B. a is greater than b

C. a and b are equal

D. comparison failed

M066. A reversed comparator for descending ints can use…

A. a.compareTo(b)

B. b.compareTo(a)

C. a.hashCode()

D. Math.abs(a-b)

Heaps, Heapsort & P05

M067. A min-heap combines…

A. BST ordering and sorted leaves

B. complete-tree shape and parent≤child ordering

C. hashing and equality

D. stack and queue modes

M068. Where must a minimum occur in a nonempty min-heap?

A. the root

B. the rightmost leaf

C. any interior node except root

D. last array position

M069. A maximum in a min-heap is guaranteed to appear at least once…

A. at the root

B. at a leaf

C. at the left child of root

D. at array index 1

M070. When sifting down a too-large node, swap with…

A. larger child

B. smaller child

C. always left child

D. always right child

M071. For a 0-based heap array, left and right children of i are…

A. i-1 and i+1

B. 2i+1 and 2i+2

C. 2i and 2i+1

D. i/2 and i+1

M072. Building a heap bottom-up by heapify takes…

A. O(n)

B. O(n log n) always

C. O(log n)

D. O(n^2)

M073. SortingMachine in insertion mode permits…

A. removeFirst only

B. add; extraction has not begun

C. removeFirst and add interleaved

D. sorting only strings

M074. SortingMachine contents abstractly form a…

A. set with no duplicates

B. multiset allowing duplicates

C. map of index to value

D. linked list only

Linked Lists, Iterators & P06

M075. Random access to an index in a singly linked list typically takes…

A. O(1)

B. O(n)

C. O(log n)

D. O(n log n)

M076. What is the purpose of smart/sentinel nodes?

A. hold default user data

B. avoid special cases at boundaries

C. keep values sorted

D. prevent garbage collection

M077. A doubly linked node ordinarily stores…

A. data, next, prev

B. data only

C. hash, index

D. left, right, parent only

M078. For a linked implementation of List, advance moves the cursor…

A. one place from right to left

B. one place from left to right

C. to end immediately

D. to start immediately

M079. List’s abstract mathematical model is…

A. (left sequence, right sequence)

B. a set

C. a map

D. a binary search tree

M080. retreat requires…

A. right nonempty

B. left nonempty

C. both partitions empty

D. sorted entries

M081. P06 uses a doubly-linked representation primarily to make retreat…

A. O(n)

B. O(log n)

C. O(1)

D. undefined

M082. An iterator’s hasNext() should…

A. remove the next entry

B. report whether unseen entries remain

C. always return true

D. sort the remaining nodes

M083. The practice Queue3 stores nodes in reverse; abstract front is adjacent to…

A. preFront

B. postRear

C. middle node

D. null

BugsWorld & BL Programs

M084. In BL, a WHILE statement body is represented in the AST as…

A. one BLOCK child

B. two CALL children

C. no children

D. a Map

M085. What is the main role of a BL compiler?

A. execute bugs directly in Java

B. translate source into representation/object code

C. generate random test inputs

D. count words

M086. A BL custom instruction is declared under…

A. INSTRUCTION name IS ... END name

B. FOR EACH name

C. CLASS name

D. private static void

M087. The token next-is-enemy is used as…

A. a condition

B. a custom instruction name only

C. a Java string literal

D. an array index

Statement ASTs & Recursion

M088. A newly constructed Statement is…

A. CALL skip

B. empty BLOCK

C. IF true

D. null

M089. Which Statement kind has two BLOCK children?

A. CALL

B. WHILE

C. IF_ELSE

D. IF

M090. A CALL leaf stores…

A. instruction name

B. a condition and three children

C. a Map of programs

D. a heap array

M091. After s3.addToBlock(0,s1), what happens to s1?

A. it is duplicated

B. it becomes an empty BLOCK

C. it becomes null

D. s3 becomes empty

M092. For an IF statement, disassembleIf(body) returns…

A. a Statement name

B. a Condition while storing its BLOCK body in body

C. the entire Program

D. a boolean execution result

M093. The correct case for counting primitive CALL occurrences is…

A. increment for each BLOCK

B. increment for each primitive CALL only

C. increment for all CALLs including custom

D. always return zero

M094. In recursive prettyPrint, indentation should increase…

A. when entering a nested body BLOCK

B. when printing every CALL

C. once per character

D. only after ELSE ends

M095. Why reassemble after recursive disassembly in read-only analyses?

A. to satisfy restoration and avoid altering input

B. to sort statements

C. to keep the object empty

D. to avoid recursion

M096. For renameInstruction, which leaf type changes?

A. BLOCK

B. CALL matching oldName

C. WHILE condition

D. IF condition

Program Kernels & Refactoring

M097. Program2 represents a Program with…

A. name String, context Map, body Statement

B. only a Statement

C. a heap of tokens

D. only an array

M098. A Program context stores…

A. primitive instructions only

B. user-defined instruction names mapped to Statement bodies

C. Java exception handlers

D. token positions

M099. Program body’s kind must be…

A. CALL

B. BLOCK

C. WHILE

D. IF

M100. swapContext(context) is used to…

A. rename a variable without effects

B. exchange the Program context with the argument Map

C. remove a single key

D. return Program name

M101. A Program-wide rename must update…

A. only main body calls

B. only the context key

C. definition key plus all corresponding CALL occurrences

D. all condition enums

M102. To count skips in a Program, recurse over…

A. just name String

B. every context Statement and the main body

C. only main body

D. only Map keys

Grammars, Tokenizer & Recursive Descent

M103. A terminal in a context-free grammar is…

A. an expandable variable

B. a literal token appearing in generated strings

C. a Java object

D. a recursive method

M104. The grammar actionlist → action | action actionlist permits…

A. zero actions only

B. one or more actions

C. exactly two actions

D. exactly zero or two

M105. A maximal token from nextWordOrSeparator continues while…

A. characters remain in the same separator/nonseparator class

B. characters are digits only

C. characters match first word

D. a newline is encountered

M106. The HW26 SEPARATORS constant contains…

A. only punctuation

B. space, tab, newline, carriage return

C. all letters

D. only spaces

M107. The token Queue is terminated by…

A. a null pointer

B. END_OF_INPUT token

C. a number 0

D. a closing curly brace

M108. In a recursive descent arithmetic parser, expr handles…

A. * / only

B. + - between terms

C. single digits only

D. parentheses only

M109. In that parser, term handles…

A. * / between factors

B. + - between expressions

C. whole Program

D. IF blocks

M110. A factor is…

A. (expr) or digit-seq

B. only digits of length 1

C. only + terms

D. only a separator

M111. Evaluate 2+3*4 with correct grammar precedence.

A. 20

B. 14

C. 24

D. 9

M112. The signed-real-const grammar allows sign as…

A. mandatory minus only

B. optional + or - at the start

C. only E sign

D. invalid always

M113. Why does HW28 use a sentinel after the input expression?

A. to mark end so charAt(0) remains safe during parsing

B. to add an extra multiplication

C. to sort terms

D. to terminate Java strings

Projects & Debugging Scenarios

M114. In Project 3 Map4, a negative hash code must be normalized because…

A. hashCode returns Strings

B. Java % can return a negative remainder and index outside the array

C. hash tables cannot contain keys

D. Map2 buckets are sorted

M115. In P07 Statement2, addToBlock transfers a child's…

A. data as a string copy

B. tree representation

C. hash value only

D. program name

M116. In P07, assembleIfElse must put children into indices…

A. both at 0

B. then 0, else 1

C. else 0, then 1

D. no children

M117. Your NaturalNumber3 isZero implementation checks…

A. this.rep.length()==0

B. this.rep.equals("0")

C. rep==null

D. a cached boolean

M118. Your Map4 iterator moves to the next bucket when…

A. current bucket iterator has no next

B. every pair value is null

C. size grows

D. hashCode changes

M119. If a JUnit test relies on an incorrect equals implementation to check add, what can happen?

A. a correct add is falsely reported incorrect

B. the compiler rejects all tests

C. add automatically sorts

D. all tests must pass

Exam-Style Tracing & Complexity

M120. With array-based min-heap [2,5,6,8,7], removing root uses which replacement before siftDown?

A. 5
B. 6
C. 7
D. 8

M121. A complete binary tree with n nodes uses an array without gaps in…

A. inorder order
B. level order
C. postorder order
D. alphabetic order

M122. A Queue in reverse doubly-linked representation has <5>; after enqueue(7), real-node order from preFront is…

A. 5,7
B. 7,5
C. 7 only
D. 5 only

M123. After s.addToBlock(0, call), the variable call normally contains…

A. the original call
B. an empty BLOCK
C. an empty CALL
D. null

M124. After adding 9,3,3 to SortingMachine in insertion mode its abstract contents are…

A. {3,9}
B. {3,3,9}
C. <3,3,9> only
D. {9}

M125. Which task must visit all nodes in an arbitrary BinaryTree in the worst case?

A. BST membership of guaranteed root
B. counting all CALL nodes
C. reading min-heap root
D. checking object identity

Part II — Short Response

Use complete reasoning, pseudocode, diagrams or code where appropriate. Model answers are in Part IV.

Contracts & Mathematical Models

S001. Explain the difference between @updates, @restores, @replaces, and @clears.

S002. Interpret this contract: @requires |q| > 0, @ensures q = tail(#q) and <result> is prefix of #q.

S003. How does a mathematical postcondition differ from Java integer arithmetic?

Sequences, Smooth & Arithmetic

S004. Compute smooth(<-5, 2, 7, 8>) with Java integer division.

S005. Design five tests for average(int,int) and explain what each covers.

S006. Explain how to write recursive versus iterative smooth while preserving the original input.

Kernel Implementations & Testing

S007. Design Queue kernel methods using a Sequence field in logical order.

S008. What should a test plan cover for a Map kernel?

S009. Define kernel purity and explain why it matters in implementation classes.

Stacks, Queues & Layering

S010. Describe an O(1) push on a singly linked stack.

S011. Explain how a queue-based Set might perform add and remove.

S012. Explain why a queue front method can be layered using only queue operations.

P01 WordCounter

S013. Trace case-insensitive counts for “Cat cat DOG dog!” using whitespace and punctuation separators.

S014. Explain WordCounter’s major pipeline from text file to output.

P02 NaturalNumber on String

S015. After rep="408", what do divideBy10 and then multiplyBy10(7) do?

S016. Give the core invariant for NaturalNumber-on-String.

Hashing & Map4

S017. Explain the Map4 convention for hashTable and size.

S018. How do Map4 add, remove, hasKey, and value use hashing?

S019. Why can a faulty hasKey make an add test appear to fail?

S020. Describe a robust hashCode design for a formatted phone number.

Trees & Recursive Traversal

S021. Show the generic recursive pattern for computing Tree size without its size kernel method.

S022. Explain the difference between height, size and number of leaves.

S023. Compute prefix representation of root a with left child b and empty right.

Binary Search Trees & P04

S024. Describe BST removal for nodes with zero, one, and two children.

S025. Contrast the practice exam’s ≤ duplicate rule with HW14’s unique-key BST invariant.

S026. Why is the balanced average BST complexity O(log n) but worst-case O(n)?

Sorting & Comparators

S027. Trace insertion sort for <4,1,3,2>.

S028. Partition <6,3,8,6,1> around 6 using <= in front.

S029. Explain the quicksort base case and how partitions are recombined.

Heaps, Heapsort & P05

S030. Give a valid min-heap for 4,6,9,11,8 and its array.

S031. Describe removeFirst in an array min-heap.

S032. Distinguish heap ordering property from heap shape property.

S033. What happens to entries after SortingMachine5 switches from insertion to extraction?

Linked Lists, Iterators & P06

S034. For a reversed doubly-linked Queue, describe enqueue(x) pointer assignments.

S035. For a reversed doubly-linked Queue, describe dequeue pointer assignments.

S036. Show List retreat transforming (<1,2>,<3,4>).

BugsWorld & BL Programs

S037. Sketch a BL loop that infects an enemy ahead, otherwise moves if empty.

S038. Distinguish BL compiler, server, client, and display.

Statement ASTs & Recursion

S039. Trace Statement transfer in assembleIfElse(c,thenBlock,elseBlock).

S040. Write the recursive case analysis for countSkips.

S041. What nodes appear in an AST for WHILE true DO move END WHILE?

S042. What does a BLOCK statement represent?

Program Kernels & Refactoring

S043. How do you restore a Program after recursively analyzing its context and body?

S044. Describe a correct Program-wide rename in terms of the context Map.

S045. Explain newContext/newBody versus swapContext/swapBody.

Grammars, Tokenizer & Recursive Descent

S046. List nonterminals and terminals for message→mascot actionlist; mascot→Brutus|Buckeyes; actionlist→action|action actionlist; action→runs|cheers.

S047. Derive Brutus cheers runs using the grammar above.

S048. Describe the algorithm for nextWordOrSeparator(text, position).

S049. Explain why recursive descent gives * higher precedence than +.

S050. Evaluate (8-3)*2+5 using expr/term/factor grammar.

S051. What causes a tokenizer to ignore whitespace but keep words?

Projects & Debugging Scenarios

S052. Compare the representation fields in P02, P03, and P07.

S053. What three categories of tests are essential for P07 Statement2 assembly/disassembly?

S054. Identify a potential defect in the uploaded HW14 BST isInTree solution and explain correction.

Exam-Style Tracing & Complexity

S055. Draw one valid complete min-heap containing 5,7,9,11 and give its level-order array.

S056. Explain why restoring a BinaryTree or Statement after a helper call matters on an exam.

Part III — Multiple Choice Key

Correct option and the reason it is correct.

Contracts & Mathematical Models

M001 — B. The # prefix denotes the value at method entry.

M002 — C. Replaces permits overwriting the old value; the postcondition describes the new value.

M003 — B. Queue values are sequences; insertion order matters and duplicates are allowed.

M004 — B. A Map models a partial function: each key has at most one corresponding value.

M005 — B. Client must establish requires; behavior otherwise is not specified by the contract.

M006 — B. Clears means the formal parameter is left at its type’s initial value.

M007 — B. Permutation preserves elements including duplicate counts.

M008 — C. The star operator is concatenation for string/sequence models.

Sequences, Smooth & Arithmetic

M009 — A. Adjacent averages: (2+4)/2=3 and (4+6)/2=5.

M010 — C. There are no adjacent pairs, so the output length is |s|-1 = 0.

M011 — C. Extreme values expose overflow; opposite signs expose truncation pitfalls.

M012 — B. Even when the mathematical average fits int, the intermediate sum may not.

M013 — B. Java integer division truncates toward zero, not toward minus infinity.

M014 — C. Test distinct contract partitions, including boundaries and exceptional conditions.

Kernel Implementations & Testing

M015 — B. A component’s secondary layer is defined in terms of its kernel and standard operations.

M016 — A. Convention specifies legal private-field states.

M017 — B. Correspondence is the abstraction function from representation to model.

M018 — A. The abstract front is index 0.

M019 — B. newInstance constructs the same dynamic type with its initial abstract value.

M020 — B. transferFrom transfers the abstract value and clears the source.

M021 — B. Specification-based partitions are systematic evidence, not a mathematical proof.

M022 — B. Kernel methods depend on one another for precondition checks and derived operations.

Stacks, Queues & Layering

M023 — B. Push/pop work at the same top end, last in first out.

M024 — B. Enqueue goes to rear, dequeue removes front.

M025 — B. Moving the boundary is implemented through stack transfers.

M026 — B. A layered method must preserve the queue’s abstract value.

M027 — A. Both alter only the head reference.

M028 — C. Sets contain at most one occurrence of each value.

P01 WordCounter

M029 — B. Lowercase normalization merges Word and word under one key.

M030 — B. Map<String,Integer> represents frequencies.

M031 — B. Map iteration is not necessarily sorted; sorting the Queue provides display order.

M032 — C. A separator-only token should not be counted as a word.

M033 — B. Characters such as < and & need HTML escaping for safe valid output.

P02 NaturalNumber on String

M034 — B. The convention forbids leading zeroes and correspondence uses empty string for 0.

M035 — B. Decimal multiply-by-ten plus digit k appends a base-10 digit.

M036 — B. It removes and returns the rightmost digit.

M037 — A. Zero is represented by the empty string; "0" would violate canonical form.

M038 — B. The destination adopts the value; the source returns to zero representation.

Hashing & Map4

M039 — B. Java % can be negative; floorMod or conditional adjustment gives valid range, even MIN_VALUE.

M040 — B. A hash index can be shared by distinct keys; bucket structure handles collisions.

M041 — A. The uploaded Map4 uses Map<K,V>[] hashTable and size.

M042 — B. Each bucket must be an actual Map before delegation.

M043 — A. It scans for a bucket with entries then removes one pair.

M044 — B. Prime sizes can improve distribution for some hash functions but guarantee neither collision freedom nor O(1) worst case.

M045 — C. Adjust -2 by +5 to get 3.

M046 — B. Java equality/hashCode contract demands equal objects have equal hashCodes.

Trees & Recursive Traversal

M047 — B. Binary trees have left and right subtrees.

M048 — A. Count root once and recursively count each child.

M049 — B. The root contributes a level; the tallest child determines total height.

M050 — B. Unless the contract updates the tree, restoration is necessary.

M051 — C. HW13 uses () for the empty tree.

M052 — B. A nonempty tree: root + (left + right); both subtrees empty.

M053 — B. BST ordering directs search to at most one child at each step.

Binary Search Trees & P04

M054 — A. Unique-key BST invariant places smaller labels to the left.

M055 — B. Leftmost node of right subtree is the next larger label.

M056 — C. In a chain-shaped BST, search may visit every node.

M057 — B. 20<50 and 20<30.

M058 — B. Ordered BST labels let membership follow one branch per node.

M059 — A. Use the minimum from the right subtree, preserving BST order.

Sorting & Comparators

M060 — A. A total preorder gives a coherent comparison relation.

M061 — B. It grows a sorted collection one element at a time.

M062 — B. The resulting queue is sorted and contains all old entries plus the inserted one.

M063 — B. Entries no larger than pivot go in front; larger go in back.

M064 — B. Balanced pivots give n log n; very skewed partitions can yield quadratic time.

M065 — A. The Comparator contract returns a negative result when a precedes b.

M066 — B. Comparing the arguments in reverse flips the order safely.

Heaps, Heapsort & P05

M067 — B. Heap definition requires complete shape and local order.

M068 — A. Every parent ≤ descendant implies root is minimum.

M069 — B. A greatest interior value must persist down to a leaf if duplicates exist.

M070 — B. The smallest child should move upward to satisfy parent≤both children.

M071 — B. Array level order indexes children at 2i+1 and 2i+2.

M072 — A. The total work over nodes by height is linear.

M073 — B. Add before switching; removeFirst after changing to extraction mode.

M074 — B. Ordering is distinct from multiplicity; duplicates remain.

Linked Lists, Iterators & P06

M075 — B. Need to follow references from a known starting node.

M076 — B. Sentinels simplify updates at empty/head/tail boundaries.

M077 — A. Next and prev allow constant-time local insertion and removal.

M078 — B. List is a pair (left,right); advance transfers rightFront to end of left.

M079 — A. A List has a cursor dividing left and right strings.

M080 — B. There must be an element before the cursor to move back.

M081 — C. A prev pointer allows constant-time one-node movement.

M082 — B. hasNext only checks availability; next obtains the next element.

M083 — B. The correspondence reads nodes backward starting at postRear.prev.

BugsWorld & BL Programs

M084 — A. WHILE has one BLOCK body child.

M085 — B. It translates source instructions into a compiled form for the VM client.

M086 — A. A user-defined instruction has a named declaration and body.

M087 — A. It tests the state of the cell immediately ahead.

Statement ASTs & Recursion

M088 — B. Initial Statement is an empty BLOCK with zero children.

M089 — C. IF_ELSE has thenBody and elseBody.

M090 — A. CALL is labeled by an instruction identifier and has no children.

M091 — B. The child statement is transferred, and its variable is cleared.

M092 — B. The condition is returned and body is transferred out.

M093 — B. Classify calls using the built-in instruction set.

M094 — A. Nested body statements use offset+INDENT_SIZE.

M095 — A. The method should preserve the incoming abstract value.

M096 — B. Only CALL names matching oldName are replaced.

Program Kernels & Refactoring

M097 — A. The uploaded implementation uses these three fields.

M098 — B. Context is a Map<String,Statement> of definitions.

M099 — B. The Program convention requires its body to be a BLOCK.

M100 — B. Swapping moves abstract context values between objects.

M101 — C. A definition rename changes context domain and calls in definitions and body.

M102 — B. Both named instruction definitions and main body contain CALLs.

Grammars, Tokenizer & Recursive Descent

M103 — B. Only nonterminals appear as LHS production symbols; terminals are emitted.

M104 — B. The base action produces one; recursion extends to arbitrarily many.

M105 — A. Token length is extended until character class changes or input ends.

M106 — B. HW26 defines separators as whitespace " \t\n\r".

M107 — B. Tokenizer contract appends END_OF_INPUT.

M108 — B. Grammar: expr→term{(+|-)term}.

M109 — A. Grammar: term→factor{(*|/)factor}.

M110 — A. The factor production handles nested parentheses or integers.

M111 — B. term computes 3*4=12 before expr adds 2.

M112 — B. signed-real-const→real-const | +real-const | -real-const.

M113 — A. The source StringBuilder has at least one trailing marker after valid expression.

Projects & Debugging Scenarios

M114 — B. For negative a, a%b can be negative; mod guarantees 0≤index<capacity.

M115 — B. The implementation adds localS.rep into the receiver’s child sequence, then clears localS.

M116 — B. Then branch precedes else branch in child order.

M117 — A. Zero’s representation is the empty string.

M118 — A. It advances currentBucket until an iterator has entries.

M119 — A. Dependent methods can make tests falsely fail; independent test observations help isolate faults.

Exam-Style Tracing & Complexity

M120 — C. Move last active array entry 7 to root before sifting it down.

M121 — B. Breadth-first level order directly maps a complete tree into a compact array.

M122 — B. Abstract queue <5,7> is stored physically 7 then 5.

M123 — B. Transfer clears source Statement.

M124 — B. Contents are a multiset and both 3 values remain.

M125 — B. Counting all matching nodes requires exploring each subtree.

Part IV — Short Response Models

Use these to assess whether you have articulated the key steps. Alternative correct approaches may exist.

Contracts & Mathematical Models

S001. Explain the difference between @updates, @restores, @replaces, and @clears.

@updates allows a specified change; @restores requires the same abstract value on return; @replaces overwrites incoming value; @clears leaves the initial value.

S002. Interpret this contract: @requires |q| > 0, @ensures q = tail(#q) and <result> is prefix of #q.

The queue must be nonempty; the method removes the first entry and returns that entry. It leaves all remaining elements in order.

S003. How does a mathematical postcondition differ from Java integer arithmetic?

A math postcondition uses exact mathematical integers, with no Java overflow. Java int arithmetic wraps on overflow and divides by truncating toward zero.

Sequences, Smooth & Arithmetic

S004. Compute smooth(<-5, 2, 7, 8>) with Java integer division.

<-1, 4, 7>: (-5+2)/2=-1; (2+7)/2=4; (7+8)/2=7.

S005. Design five tests for average(int,int) and explain what each covers.

(MAX,MAX) overflow; (MIN,MIN) overflow; (MAX,MIN) mixed signs; (-5,-8) negative rounding; (3,4) odd positive sum.

S006. Explain how to write recursive versus iterative smooth while preserving the original input.

Iterative: loop adjacent indices and append each average. Recursive: base case length <=1, average first two then recursively smooth suffix; restore any removed entries before return.

Kernel Implementations & Testing

S007. Design Queue kernel methods using a Sequence field in logical order.

enqueue appends at sequence.length(); dequeue removes index 0; length returns sequence.length(). Keep correspondence as abstract Queue equals sequence contents.

S008. What should a test plan cover for a Map kernel?

Constructor/empty map, add unique key, remove present key, removeAny nonempty, value present key, hasKey both outcomes, size transitions, and same/different hash-bucket cases.

S009. Define kernel purity and explain why it matters in implementation classes.

Kernel methods should be implemented directly from representation and permitted primitives rather than relying on secondary methods of the same component; this avoids circular dependencies.

Stacks, Queues & Layering

S010. Describe an O(1) push on a singly linked stack.

Create a node with x, set its next to the current top, update top to new node, and increment length.

S011. Explain how a queue-based Set might perform add and remove.

add checks membership then enqueues if absent; remove rotates/searches for the element and retains all other entries. Correctness of no duplicates is required.

S012. Explain why a queue front method can be layered using only queue operations.

Dequeue saves front, re-enqueue it, and rotate remaining entries as needed so the original sequence is restored; return the saved front.

P01 WordCounter

S013. Trace case-insensitive counts for “Cat cat DOG dog!” using whitespace and punctuation separators.

{cat:2, dog:2}. Distinct punctuation groups are discarded; lowercase normalizes capitalization.

S014. Explain WordCounter’s major pipeline from text file to output.

Read input lines, split words/separators, normalize words, count with Map, collect/sort keys in Queue, emit escaped HTML table rows using sorted words.

P02 NaturalNumber on String

S015. After rep="408", what do divideBy10 and then multiplyBy10(7) do?

divideBy10 returns 8 and leaves "40"; multiplyBy10(7) makes "407".

S016. Give the core invariant for NaturalNumber-on-String.

Representation is a string of digits with no leading zero; zero has the canonical empty-string representation.

Hashing & Map4

S017. Explain the Map4 convention for hashTable and size.

Array length positive, every bucket nonnull, each key resides in bucket mod(key.hashCode,capacity), size equals sum of bucket sizes.

S018. How do Map4 add, remove, hasKey, and value use hashing?

Compute nonnegative index from key hash then delegate to that bucket; add/remove also update global size as specified.

S019. Why can a faulty hasKey make an add test appear to fail?

add asserts key absence via hasKey. A defect there can trigger an assertion before the add body runs.

S020. Describe a robust hashCode design for a formatted phone number.

Combine digits in a deterministic way that gives equal numbers identical hashes, consider overflow, and keep it consistent with equals; never assume hash values uniquely identify objects.

Trees & Recursive Traversal

S021. Show the generic recursive pattern for computing Tree size without its size kernel method.

If empty return 0; disassemble into root and child sequence, answer=1+sum(size(child)), reassemble root and children, return answer.

S022. Explain the difference between height, size and number of leaves.

Size counts all nodes. Height counts nodes on the longest root-to-leaf path (empty 0 in HW21). Leaves count nodes with zero children.

S023. Compute prefix representation of root a with left child b and empty right.

a(b(()())())

Binary Search Trees & P04

S024. Describe BST removal for nodes with zero, one, and two children.

Zero: remove leaf. One: replace node with its only child. Two: replace label with inorder successor from right subtree (or predecessor variant if specified) and remove successor from old location.

S025. Contrast the practice exam's ≤ duplicate rule with HW14's unique-key BST invariant.

The practice exam allows duplicate labels and sends equality left; HW14 defines IS_BST with no duplicate labels. Apply each problem's exact contract.

S026. Why is the balanced average BST complexity O(log n) but worst-case O(n)?

Balanced tree height grows logarithmically; arbitrary insertion order may create a chain of height n.

Sorting & Comparators

S027. Trace insertion sort for <4,1,3,2>.

Sorted temporary queue grows: <4>, <1,4>, <1,3,4>, <1,2,3,4>. Original input is consumed then replaced.

S028. Partition <6,3,8,6,1> around 6 using <= in front.

front=<6,3,6,1>, back=<8>, if processing queue left-to-right and preserving order.

S029. Explain the quicksort base case and how partitions are recombined.

If queue length ≤1, return. Otherwise dequeue pivot, partition remainder into ≤ and > queues, recursively sort each, then concatenate front, pivot, back.

Heaps, Heapsort & P05

S030. Give a valid min-heap for 4,6,9,11,8 and its array.

One example tree root 4; children 6 and 9; 6 children 11 and 8. Array [4,6,9,11,8].

S031. Describe removeFirst in an array min-heap.

Save root, move last active element to index 0, decrement heapSize, recursively/iteratively sift down by smaller child, return saved minimum.

S032. Distinguish heap ordering property from heap shape property.

Ordering: every node≤its children. Shape: complete binary tree. HW17 explicitly asks to check ordering independent of shape.

S033. What happens to entries after SortingMachine5 switches from insertion to extraction?

Queue entries are moved to an array-based heap and emptied; heapSize tracks the active prefix; insertionMode becomes false.

Linked Lists, Iterators & P06

S034. For a reversed doubly-linked Queue, describe enqueue(x) pointer assignments.

Insert immediately after preFront: new.prev=preFront; new.next=oldFirst; preFront.next=new; oldFirst.prev=new; length++.

S035. For a reversed doubly-linked Queue, describe dequeue pointer assignments.

Save postRear.prev node and its data; link its prev to postRear in both directions; length--; return saved data.

S036. Show List retreat transforming (<1,2>,<3,4>).

(<1>,<2,3,4>): move the last element of left to the front of right without changing concatenation.

BugsWorld & BL Programs

S037. Sketch a BL loop that infects an enemy ahead, otherwise moves if empty.

WHILE true DO
IF next-is-enemy THEN infect ELSE IF next-is-empty THEN move END IF END IF
END WHILE (with valid nesting/formatting).

S038. Distinguish BL compiler, server, client, and display.

Compiler translates BL; server hosts simulation; client executes compiled bug behavior; display renders the simulation.

Statement ASTs & Recursion

S039. Trace Statement transfer in assembleIfElse(c,thenBlock,elseBlock).

Receiver becomes IF_ELSE with two BLOCK children. Source variables thenBlock and elseBlock are cleared to empty BLOCK. No deep copies are made.

S040. Write the recursive case analysis for countSkips.

BLOCK iterate children; IF and WHILE recurse into body; IF_ELSE recurse both branches; CALL check name.equals("skip"); restore any dissected object.

S041. What nodes appear in an AST for WHILE true DO move END WHILE?

WHILE(TRUE) → BLOCK → CALL(move).

S042. What does a BLOCK statement represent?

An ordered sequence of zero or more (non-BLOCK) statements, modeled as child subtrees.

Program Kernels & Refactoring

S043. How do you restore a Program after recursively analyzing its context and body?

swapContext out, iterate statements, swapContext back; swapBody out, recursively analyze, swapBody back.

S044. Describe a correct Program-wide rename in terms of the context Map.

Remove oldName definition if present, rename all calls in every definition and body, then add that definition under newName, preserving unrelated entries and satisfying identifier/precondition restrictions.

S045. Explain newContext/newBody versus swapContext/swapBody.

newContext/newBody create empty objects of correct types; swap operations exchange program fields with supplied objects to allow manipulation and later restoration.

Grammars, Tokenizer & Recursive Descent

S046. List nonterminals and terminals for message→mascot actionlist; mascot→Brutus|Buckeyes; actionlist→action|action actionlist; action→runs|cheers.

Nonterminals: message, mascot, actionlist, action. Terminals: Brutus, Buckeyes, runs, cheers.

S047. Derive Brutus cheers runs using the grammar above.

message→mascot actionlist→Brutus action actionlist→Brutus cheers actionlist→Brutus cheers action→Brutus cheers runs.

S048. Describe the algorithm for nextWordOrSeparator(text, position).

Classify text.charAt(position) as separator/nonseparator, increment end while characters have the same class, return substring(position,end).

S049. Explain why recursive descent gives * higher precedence than +.

valueOfExpr invokes valueOfTerm for complete multiplicative chunks; terms are evaluated before the expr level combines them by + or -.

S050. Evaluate (8-3)*2+5 using expr/term/factor grammar.

15: parenthesized factor 8-3=5, multiplied by 2 gives 10, then +5 gives 15.

S051. What causes a tokenizer to ignore whitespace but keep words?

Whenever the maximal token starts with a separator, skip it; otherwise enqueue it. Append END_OF_INPUT after all lines.

Projects & Debugging Scenarios

S052. Compare the representation fields in P02, P03, and P07.

P02 NaturalNumber3: String rep. P03 Map4: array of Map buckets plus size. P07 Program2: name String, Map context, Statement body; Statement2: Tree<StatementLabel> rep.

S053. What three categories of tests are essential for P07 Statement2 assembly/disassembly?

Empty BLOCK and boundary child positions; nested IF/IF_ELSE/WHILE/CALL round trips; transfer/clearing behavior of source statements and restoration after disassembly.

S054. Identify a potential defect in the uploaded HW14 BST isInTree solution and explain correction.

Its shown branch when root>key recurses right; BST search should recurse left when root>key and right when root<key. It also needs to reassemble after disassembly to restore the tree.

Exam-Style Tracing & Complexity

S055. Draw one valid complete min-heap containing 5,7,9,11 and give its level-order array.

Root 5, children 7 and 9, left child of 7 is 11. Array [5,7,9,11].

S056. Explain why restoring a BinaryTree or Statement after a helper call matters on an exam.

The method’s @restores/ensures contract often requires the input abstract value unchanged; disassembly temporarily empties/transfers values, so reassembly is essential.

Companion app: CSE2231_Midterm_Studio.zip · Run python app.py with JDK 17+.